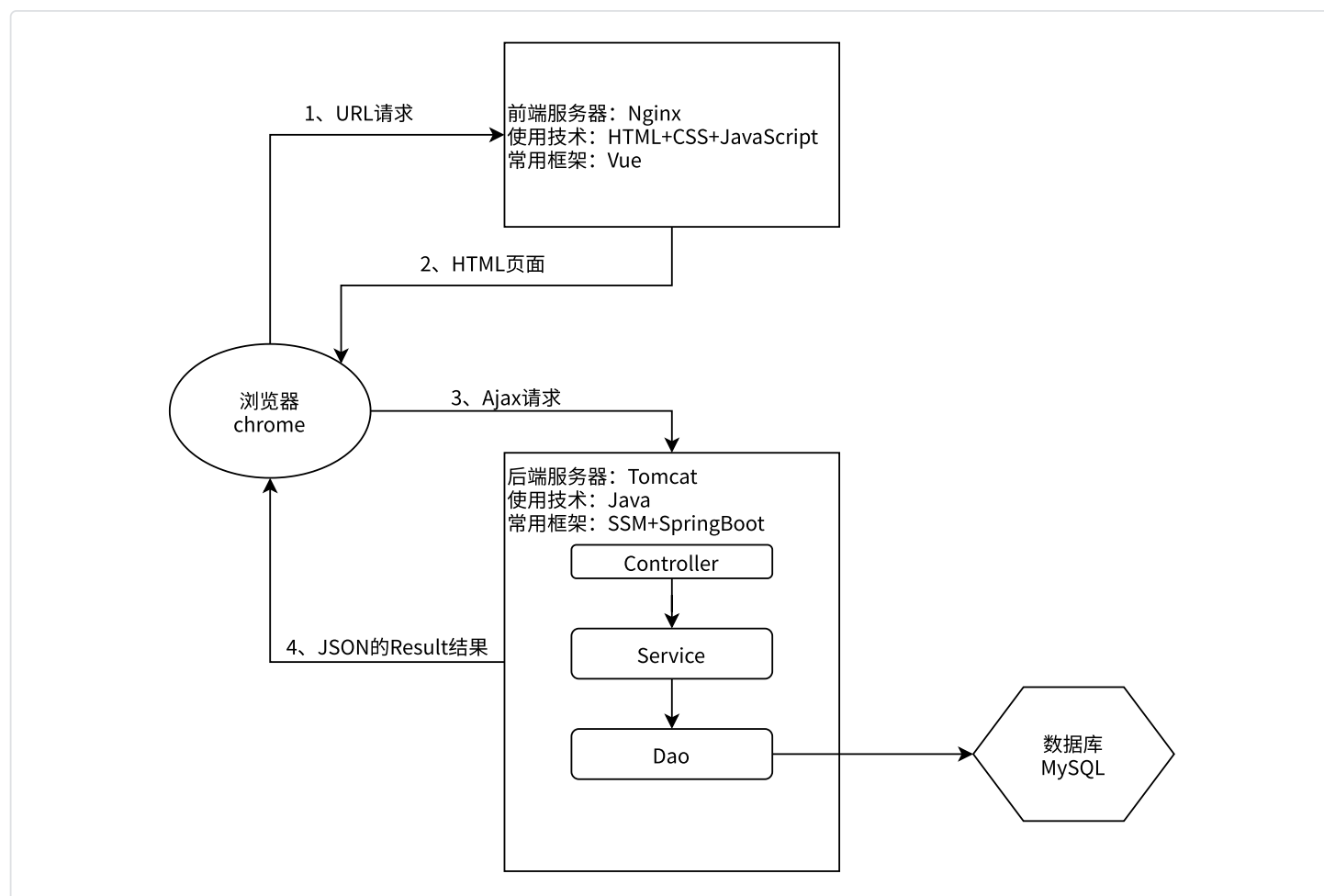


6-SpringBoot项目涉及知识点

一.基于前后端分离的BS开发模型

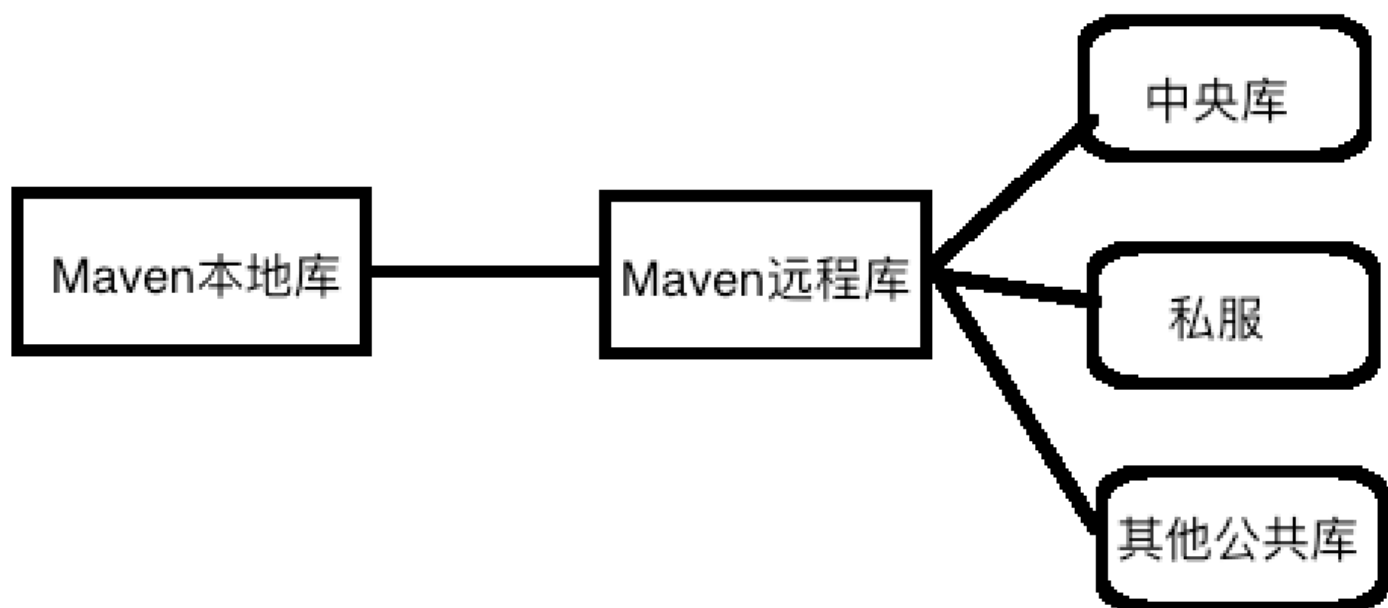
该项目我们使用SpringBoot开发后端，并提供对外的API接口。



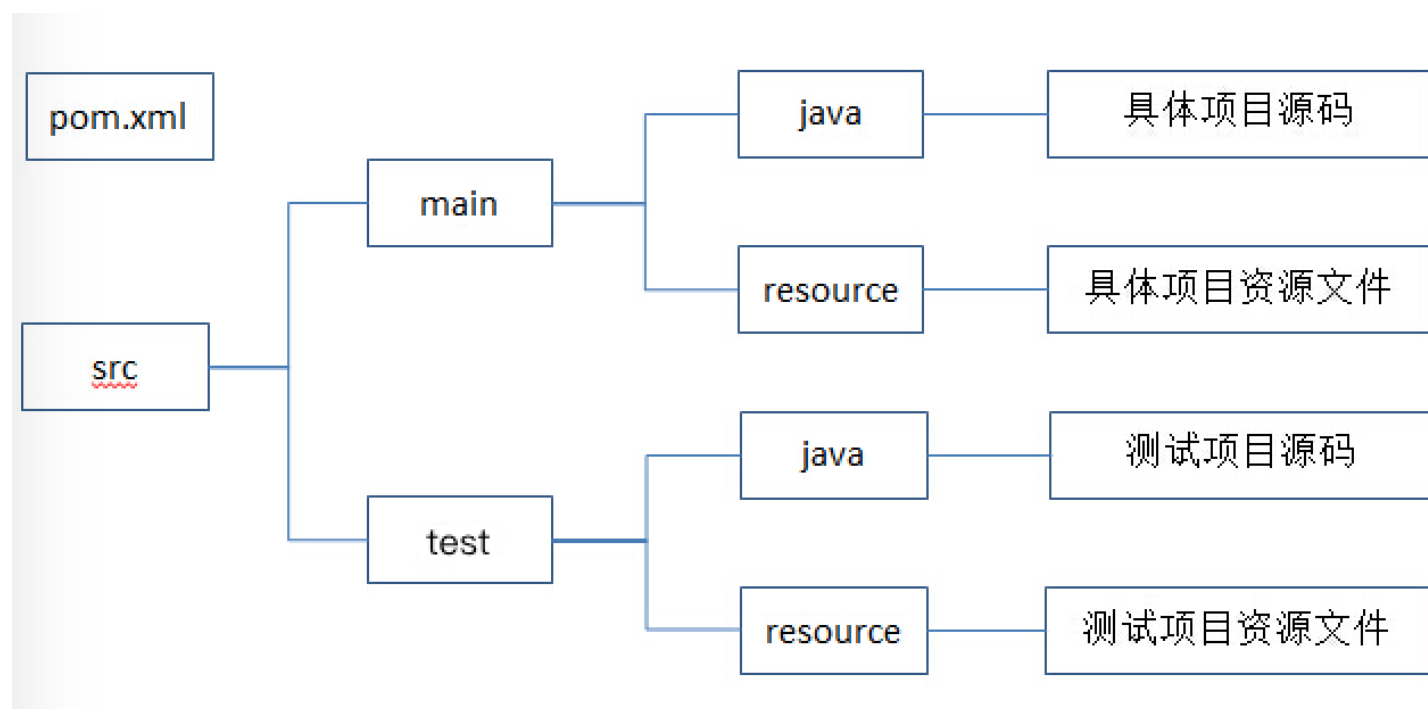
跨域问题及解决方案

二.Maven简介

1.Maven原理



2.Maven项目目录结构



三.HTTP请求方式和数据传输方式

1.HTTP的15种请求方式：

	A	B	C
1	序号	方法	描述
2	1	GET	请求指定的页面信息，并返回实体主体。
3	2	HEAD	类似于get请求，只不过返回的响应中没有具体的内容，用于获取报头
4	3	POST	向指定资源提交数据进行处理请求（例如提交表单或者上传文件）。数 求体中。POST请求可能会导致新的资源的建立和/或已有资源的修改。
5	4	PUT	从客户端向服务器传送的数据取代指定的文档的内容。
6	5	DELETE	请求服务器删除指定的页面。
7	6	CONNECT	HTTP/1.1协议中预留给能够将连接改为管道方式的代理服务器。
8	7	OPTIONS	允许客户端查看服务器的性能。
9	8	TRACE	回显服务器收到的请求，主要用于测试或诊断。
10	9	PATCH	实体中包含一个表，表中说明与该URI所表示的原内容的区别。
11	10	MOVE	请求服务器将指定的页面移至另一个网络地址。
12	11	COPY	请求服务器将指定的页面拷贝至另一个网络地址。
13	12	LINK	请求服务器建立链接关系。
14	13	UNLINK	断开链接关系。
15	14	WRAPPED	允许客户端发送经过封装的请求。
16	15	Extension-mothed	在不改动协议的前提下，可增加另外的方法。

2.HTTP数据传输方式

一、表单数据

传统的，通过表单的形式由浏览器将数据发送到服务器

- 发送的类型为：application/x-www-form-urlencoded
- 数据格式为：userName=david&password=123456
- controller中直接接收数据。

二、JSON数据

Ajax中常用的发送方式，发送JSON类型数据。

- 发送的类型为：application/json
- 数据格式为：{"userName":"david","password":"123456"}
- controller中需要添加注解@RequestBody接收数据。

三、Restful风格数据

Restful API接口规范

四.postman测试工具使用

1.添加

- 通常使用 **POST** 方式添加数据。
- body 选择为 **raw**，类型选 **JSON**。
- 内容格式为JSON。
- response中显示插入后返回结果。

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** http://localhost:3000/posts
- Body Type:** raw (selected), JSON (dropdown)
- Body Content:**

```
1 [{"title": "java基础", "author": "david"}]
```
- Response:** Status: 201 Created, Time: 48 ms, Size: 542 B
- Response Body (Pretty):**

```
1 {
2   "title": "java基础",
3   "author": "david",
4   "id": 2
5 }
```

2.删除

- delete方式为删除。
- 可以使用restful风格或者JSON数据，该示例使用restful风格

The screenshot shows a REST client interface with the following details:

- Method:** DELETE
- URL:** http://localhost:3000/posts/1
- Body Type:** raw (selected), JSON (dropdown)
- Body Content:** (Empty)
- Response:** Status: 200 OK, Time: 32 ms, Size: 372 B
- Response Body (Pretty):**

```
1 {}
```

3.修改

- 修改一般用patch方式，put也可以。

- body 选择为 raw，类型选JSON。
- JSON中必须有主键ID值。。

PATCH

http://localhost:3000/posts/2

Send

Params

Authorization

Headers (11)

Body

Pre-request Script

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

{ "title": "java基础", "author": "king" }

Body

Cookies (1)

Headers (12)

Test Results

Status: 200 OK

Time: 17 ms

Size: 430 B

Sav

Pretty

Raw

Preview

Visualize

JSON

1

{

2

"title": "java基础",

3

"author": "king",

4

"id": 2

5

}

4.查询

注意：

- 一般是get方式查询
- 登录验证查询需要用post

get方式查询

会在url中显示数据，开发中一般不使用。

GET

http://localhost/user/edit?userId=2

Send

Params

Authorization

Headers (10)

Body

Pre-request Script

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

	KEY	VALUE	DESCRIPTION
☒	userid	1	
	Key	Value	Description

Body

Cookies (1)

Headers (5)

Test Results

Status: 200 OK

Time: 191 ms

Size: 313 B

Sav

Pretty

Raw

Preview

Visualize

JSON

1

{

2

"code": 200,

3

"message": "success",

4

"data": {

5

"userId": 2,

6

"userName": "david",

7

"password": "123456",

8

"email": "david@qq.com",

9

"createTime": "2022-06-14 17:02:20"

10

}

11

}

post方式查询

- 登录验证【application/json】。

POST

http://localhost/login

Send

Params

Authorization

Headers (10)

Body

Pre-request Script

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

JSON

1

[{"userName": "david", "password": "123456"}]

Body

Cookies (1)

Headers (5)

Test Results

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

6

7

8

9

10

11

"code": 200,

"message": "success",

"data": {

"userId": 2,

"userName": "david",

"password": "123456",

"email": "david@qq.com",

"createTime": "2022-06-14 17:02:20"

}

- 根据userId查询用户【application/x-www-form-urlencoded】

POST

http://localhost/user/edit

Send

Params

Authorization

Headers (10)

Body

Pre-request Script

Tests

Settings

none

form-data

x-www-form-urlencoded

raw

binary

GraphQL

	KEY	VALUE	DESCRIPTION
☒	userid	1	
	Key	Value	Description

Body

Cookies (1)

Headers (5)

Test Results

Pretty

Raw

Preview

Visualize

JSON

1

2

3

4

5

6

7

8

9

10

11

"code": 200,

"message": "success",

"data": {

"userId": 1,

"userName": "admin",

"password": "123456",

"email": "123456@qq.com",

"createTime": "2022-06-13 17:18:06"

}

- ！ 总结一下在代码开发的过程中，前后端配合开发的一些规范：
- 前端发送数据统一使用post方式，后端服务器中使用@PostMapping来处理请求。
 - 若只需要传递一个值则使用表单方式发送，后端不需要注解配合。
 - 若传递多个值使用JSON对象发送，后端需要使用@RequestBody配合。
 - 后端返回的数据都是JSON对象。

五.SpringBoot项目中常用注解

1.@Controller和@RestController

在SpringMVC 中，控制器Controller 负责处理由DispatcherServlet 分发的请求，它把用户请求的数据经过业务处理层处理之后封装成一个Model，然后再把该Model 返回给对应的View 进行展示。@Controller 用于标记在一个类上，使用它标记的类就是一个SpringMVC Controller 对象。分发处理器将会扫描使用了该注解的类的方法，并检测该方法是否使用了@RequestMapping 注解。

@Controller 只是定义了一个控制器类，而使用@RequestMapping 注解的方法才是真正处理请求的处理器。单单使用@Controller 标记在一个类上还不能真正意义上的说它就是SpringMVC 的一个控制器类，因为还需要配合@RequestMapping一起使用。

@RestController是一个组合注解，它等价于：

@RestController = @Controller+@ResponseBody

是前后端分离开发的专用注解。

2.@RequestMapping和它的衍生

RequestMapping是一个用来处理请求地址映射的注解，可用于类或方法上。用于类上，表示类中的所有响应请求的方法都是以该地址作为父路径。

RequestMapping注解有六个属性，下面我们把她分成三类进行说明（下面有相应示例）。

1、value, method;

value: 指定请求的实际地址，指定的地址可以是URI Template 模式（后面将会说明）；

method: 指定请求的method类型，GET、POST、PUT、DELETE等；

2、consumes, produces

consumes: 指定处理请求的提交内容类型（Content-Type），例如application/json, text/html;

produces: 指定返回的内容类型，仅当request请求头中的(Accept)类型中包含该指定类型才返回；

3、params, headers

params: 指定request中必须包含某些参数值是，才让该方法处理。

headers: 指定request中必须包含某些指定的header值，才能让该方法处理请求。

! Spring4.3中引进了 {@GetMapping、@PostMapping、@PutMapping、@DeleteMapping、@PatchMapping} 来帮助简化常用的HTTP方法的映射 并更好地表达被注解方法的语义。

3.@Autowired和@Resource

@Autowired为Spring提供的注解，需要导入包

org.springframework.beans.factory.annotation.Autowired，默认按照byType注入。如果想按名称注入，可以结合@Qualifier注解一起使用。

@Resource默认按照ByName自动注入，由J2EE提供，需要导入包javax.annotation.Resource。

@Resource有两个重要的属性：name和type，如果没有指定name属性，当注解标注在字段上，即默认取字段的名称作为bean名称寻找依赖对象，当注解标注在属性的setter方法上，即默认取属性名作为bean名称寻找依赖对象。需要注意的是，@Resource如果没有指定name属性，并且按照默认的名称仍然找不到依赖对象时，@Resource注解会回退到按类型装配。但一旦指定了name属性，就只能按名称装配了。

```
1 public class TestServiceImpl {
2     // 下面两种自动装配只要使用一种即可
3     //按类型
4     @Autowired
5     private UserDao userDao;
6
7     //按名字
8     @Resource(name="userDao")
9     private UserDao userDao;
10
11 }
```

4.@PathVariable（不用）

用于将请求URL中的模板变量映射到功能处理方法的参数上，即取出URL模板中的变量作为参数。通常配合Restful风格使用。如：

www.lanqiao.org?userId=100&roleId=2

```
getLogin(userId,roleId)
```

www.lanqiao.org/u/100/r/2

```

1 @Controller
2 public class TestController {
3     @RequestMapping(value="/user/{userId}/roles/{roleId}",method = RequestMethod.GET)
4     public String getLogin(@PathVariable("userId") String userId,
5         @PathVariable("roleId") String roleId){
6         System.out.println("User Id : " + userId);
7         System.out.println("Role Id : " + roleId);
8         return "hello";
9     }
10    @RequestMapping(value="/product/{productId}",method = RequestMethod.GET)
11    public String getProduct(@PathVariable("productId") String productId){
12        System.out.println("Product Id : " + productId);
13        return "hello";
14    }
15    @RequestMapping(value="/javabeat/{regex1:[a-z-]+}",
16        method = RequestMethod.GET)
17    public String getRegExp(@PathVariable("regex1") String regex1){
18        System.out.println("URI Part 1 : " + regex1);
19        return "hello";
20    }
21 }

```

5.@RequestParam (不用)

@RequestParam主要用于在SpringMVC后台控制层获取参数，类似一种是request.getParameter("name")，它有三个常用参数：defaultValue = "0", required = false, value = "isApp"; defaultValue 表示设置默认值，required 通过boolean设置是否是必须要传入的参数，value 值表示接受的传入的参数类型。

@RequestParam只用来处理Content-Type: 为 `application/x-www-form-urlencoded` 编码的内容，提交方式GET、POST

6.@ResponseBody (不用)

作用：该注解用于将Controller的方法返回的对象，通过适当的HttpMessageConverter转换为指定格式后，写入到Response对象的body数据区。

使用时机：返回的数据不是jsp页面，而是其他某种格式的数据时（如json、xml等）

7.@RequestBody

该注解常用来处理Content-Type: 不是application/x-www-form-urlencoded编码的内容，例如application/json, application/xml等。通过使用HandlerAdapter 配置的HttpMessageConverters来解析post data body，然后绑定到相应的bean上的。

因为配置有FormHttpMessageConverter，所以也可以用来处理 application/x-www-form-urlencoded的内容，处理完的结果放在一个MultiValueMap<String, String>里，一般不建议使用。

8.@Component、@Repository 和 @Service

- @Component相当于通用的注解，当不知道一些类归到哪个层时使用，但是不建议。
- @Repository 用于注解dao层，最新的mybatis支持使用@Mapper
- @Service用与服务层

! 总结一下项目开发中的常用注解：

Spring的注解

- @RestController
- @Autowired
- @GetMapping("/login")...
- @Service
- @RequestBody

Mybatis中的注解

- @Mapper
- @Select
- @Insert
- @Delete
- @Update

六.事务处理

1.事务的概念

事务 (Transaction)是由一系列对系统中数据进行访问与更新的操作所组成的一个程序 执行逻辑单元 (Unit)。事务具有四个特征，简称为事务的ACID特性。

- 原子性 (Atomicity)：原子性是指事务是一个不可分割的工作单位，事务中的操作要么都发生，要么都不发生。
- 一致性 (Consistency)：事务执行前后数据的完整性必须保持一致。比如在转账事务操作中，事务执行前后金额的总数应保持不变。
- 隔离性 (Isolation)：事务的隔离性是多个用户并发访问数据库时，数据库为每一个用户开启的事务，不能被其他事务的操作数据所干扰，多个并发事务之间要相互隔离。
- 持久性 (Durability)：持久性是指一个事务一旦被提交，它对数据库中数据的改变就是永久性的，接下来即使数据库发生故障也不应该对其有任何影响。

2.使用注解@Transactional

```

1 @Service
2 public class GeoFenceService {
3
4     @Autowired
5     private MoonlightMapper moonlightMapper;
6
7     @Transactional
8     public int addGeoFence(GeoFence geoFence) {
9         moonlightMapper.insertOne(geoFence) //创建一个事务
10        moonlightMapper.insertOne(geoFence) //将当前事务添加到已有事务中去
11        return moonlightMapper.insertOne(geoFence);
12    }
13 }

```

七.pagehepler插件使用

1、添加依赖

```

1 <dependency>
2   <groupId>com.github.pagehelper</groupId>
3   <artifactId>pagehelper-spring-boot-starter</artifactId>
4   <version>1.4.1</version>
5 </dependency>

```

2、controller代码写法

```

1 @GetMapping("getAll")
2 public Result getAll(int pageNum, int pageSize) {
3     PageHelper.startPage(pageNum, pageSize);
4     List<Message> messageList = messageService.queryALLMessage();
5     PageInfo<Message> pageInfo = new PageInfo<>(messageList);
6     return Result.success(pageInfo);
7 }

```

八.各功能API清单

1.用户管理API

- 用户登录: `/user/login`
- 列表查询: `/user/list`
- 查询用户: `/user/edit`
- 新增用户: `/user/insert`

- 编辑用户: `/user/update`
- 删除用户: `/user/delete`
- 搜索用户: `/user/search`

2.角色管理API

- 角色列表: `/role/list`
- 角色新增: `/role/insert`
- 角色编辑: `/role/edit`
- 角色删除: `/role/delete`

3.菜单管理API

- 列表查询: `/menu/getTree`
- 菜单新增: `/menu/insert`
- 菜单编辑: `/menu/edit`
- 菜单删除: `/menu/delete`